

Setup and Run Guide

Phase	Description	Output
Phase 1	Unified Dataset Pipeline	metadata.csv + splits
Phase 2	Flood / No-Flood Classifier	flood_classifier_best.pth
Phase 3	Segmentation + Risk Engine	best_model.pth
Phase 4	YOLO + Tracking + Dashboard	Live Streamlit App

System Requirements: CPU: Intel i5 or better | RAM: 16 GB minimum, 32 GB recommended | OS: Windows 10 / 11 | Python: 3.10 or 3.11 | Storage: 10 GB free

1. System Requirements

Component	Requirement
CPU	Intel i5 or better (i9-12900 recommended)
RAM	16 GB minimum, 32 GB recommended
Storage	10 GB free space for datasets and checkpoints
GPU	Not required — all code is CPU-optimized
Operating System	Windows 10 or Windows 11
Python Version	3.10 or 3.11 only (not 3.12 — PyTorch compatibility issues)

Important: Do not use Python 3.12. PyTorch has known compatibility issues with 3.12. Use Python 3.10 or 3.11 specifically.

2. Installation Steps

1 Install Python

Go to <https://www.python.org/downloads/> and download Python 3.10 or 3.11. During installation, check the box **Add Python to PATH** before clicking Install.

Verify the install by opening Command Prompt and running:

```
python --version
```

You should see *Python 3.10.x* or *Python 3.11.x*.

2 Extract the Project

Extract the downloaded **flood_victim_detection_v3.zip** to your Desktop. The project root folder should be at:

```
C:\Users\Test1\Desktop\flood_victim_detection_v3\
```

The extracted folder structure should contain:

```
flood_victim_detection_v3\  
configs\ classification\ segmentation\  
models\ risk_engine\ detection\  
data\ tracking\ pipeline\  
utils\ visualization\ video\  
training\ streamlit_app\  
checkpoints\ outputs\  
requirements.txt README.md
```

3 Set Up Dataset Folders

The project expects your datasets at a specific location. Ensure the folder structure matches exactly:

```
C:\Users\Test1\Desktop\flood_victim_detection\  
dataset 1\ (note: space in name)  
images\  
001.png  
002.png  
masks\  
001.png  
002.png  
Dataset 2\ (note: capital D)  
Flood Images\  
flood_001.jpg  
Non Flood Images\  
dry_001.jpg
```

Folder names are case-sensitive and must match exactly: "dataset 1" (with space), "Dataset 2" (capital D), "Flood Images", "Non Flood Images".

If your dataset is stored elsewhere, open **configs\config.py** and update this line to match your actual path:

```
PROJECT_ROOT = r"C:\Users\Test1\Desktop\flood_victim_detection"
```

Keep the r prefix — it handles Windows backslashes correctly.

4 Create a Virtual Environment

Open Command Prompt and navigate to the project folder:

```
cd C:\Users\Test1\Desktop\flood_victim_detection_v3
```

Create the virtual environment:

```
python -m venv venv
```

Activate it:

```
venv\Scripts\activate
```

You will see **(venv)** appear at the start of your command prompt. This confirms the environment is active. All packages you install will stay isolated inside this project and will not affect other Python projects.

5 Install Dependencies

With the virtual environment active, install all required packages:

```
pip install -r requirements.txt
```

This installs PyTorch, torchvision, ultralytics (YOLOv8), OpenCV, albumentations, Streamlit, scikit-learn, pandas, matplotlib, tqdm, and Pillow. Download may take 5 to 15 minutes. PyTorch alone is around 200 MB.

If torch installation fails, install it separately first then re-run requirements:

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
pip install -r requirements.txt
```

Verify all three critical packages installed correctly:

```
python -c "import torch; print(torch.__version__)"
python -c "from ultralytics import YOLO; print('YOLO OK')"
python -c "import streamlit; print(streamlit.__version__)"
```

All three should print version numbers without errors.

3. Phase 1 — Dataset Pipeline

Phase 1 verifies your datasets, processes all images to a unified format, generates empty masks for non-flood images, and creates train/validation/test splits. Run these three steps in order. They only need to be run once.

6 Verify Datasets

Run from the project root folder (with venv active):

```
python utils\verify_dataset.py
```

Expected output:

```
=====
VERIFYING DATASET 1
=====
Total Images : 1450
Total Masks : 1450
Missing Masks : 0
Missing Images : 0
Valid Pairs : 1448

=====
VERIFYING DATASET 2
=====
Flood Images : 800
Non Flood Images : 900
```

If **Valid Pairs** shows 0, your dataset folder paths are incorrect. Go back and update PROJECT_ROOT in configs\config.py. A non-zero **Missing Masks** count is safe — those files are skipped automatically.

7 Build Metadata

```
python data\build_metadata.py
```

This resizes all images to 256x256, copies them into the processed_data folder, creates empty masks for non-flood images (the most important upgrade in v3), and writes a unified metadata.csv. Expected to take 5 to 15 minutes.

Expected output:

```
Processing Dataset 1...
100%|██████████| 1448/1448
Processing Dataset 2 Flood Images...
100%|██████████| 800/800
Processing Dataset 2 Non Flood Images...
100%|██████████| 900/900
Metadata Saved
Total Samples : 3148
```

8 Create Train / Val / Test Splits

```
python data\create_splits.py
```

Expected output:

```
Dataset Splits Created
Train : 2203
Val : 472
Test : 473
```

This creates three CSV files in the splits\ folder: train.csv, val.csv, and test.csv. The split is stratified 70 / 15 / 15 to preserve class balance.

4. Phase 2 — Train the Flood Classifier

This trains the EfficientNet-B0 model that acts as the first gate in the pipeline. When an image is submitted to the system, this classifier runs first. If it decides NO FLOOD, the image is rejected before reaching the segmentation model, preventing false flood masks on portraits, indoor scenes, and dry roads.

9 Run Classifier Training

```
python classification\training\train_classifier.py
```

Training runs for up to 20 epochs with early stopping (patience = 5). On an i9-12900 CPU expect roughly 2 to 5 minutes per epoch.

Expected output per epoch:

```
=====
Epoch 3/20
=====
Train Loss: 0.3124: 100%|████| 69/69
Val Loss: 0.2891: 100%|████| 15/15
Train Metrics: {'accuracy': 0.88, 'f1': 0.87, 'auc': 0.93}
Validation Metrics: {'accuracy': 0.91, 'f1': 0.90, 'auc': 0.96}
Best classifier saved
```

*The best model is automatically saved to **checkpoints\flood_classifier_best.pth**. Training stops early if validation F1 does not improve for 5 consecutive epochs. A well-trained classifier should reach F1 of 0.88 to 0.94.*

5. Phase 3 — Train the Segmentation Model

This trains the Attention U-Net that segments flood regions at the pixel level. Phase 3 includes confidence suppression (prevents weak activations from becoming flood masks), morphological cleanup (removes noise and isolated blobs), and the adaptive risk engine that scores victims based on overlap, water density, segmentation confidence, and body visibility.

10 Run Segmentation Training

```
python training\train.py
```

Training runs for up to 50 epochs with early stopping (patience = 8). Expect 8 to 20 minutes per epoch on CPU. Total training time is 4 to 16 hours depending on dataset size.

Expected output per epoch:

```
-----  
Epoch 5/50 | LR: 1.00e-03  
Train | Loss=0.4821 IoU=0.5241 Dice=0.6102  
Val | Loss=0.4014 IoU=0.6108 Dice=0.7023 (612.4s)  
Best model saved (val_loss=0.4014)
```

Best model saved to **checkpoints\best_model.pth**. A good model should reach IoU of 0.65 to 0.80 after full training.

To resume training after interruption:

```
python training\train.py --resume
```

You can stop training at any time with Ctrl+C. The best checkpoint saved so far will still be used when you run the dashboard. The system loads whatever exists in checkpoints\best_model.pth.

6. Phase 4 — Launch the Dashboard

Once both checkpoints exist (flood_classifier_best.pth and best_model.pth), the full pipeline is ready. Phase 4 integrates YOLOv8 person detection, centroid tracking, adaptive risk scoring, and the Streamlit dashboard.

11 Launch the Streamlit App

Make sure you are in the project root and the virtual environment is active, then run:

```
streamlit run streamlit_app\app.py
```

Streamlit will print:

```
You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.1.x:8501
```

Your browser will open automatically. If it does not, paste **http://localhost:8501** into your browser manually.

7. Using the Dashboard

Mode	How to Use
Image Upload	Select 'Image Upload' in sidebar. Click Browse files. Upload a JPG or PNG. Results appear automatically: f
Video Upload	Select 'Video Upload'. Upload MP4, AVI, or MOV. Click Process Video. Wait for CPU processing (2-5 FPS)
Webcam	Select 'Webcam'. Click the camera button to capture a frame. The system analyses that single frame imme

Sidebar toggles: Show Flood Mask (binary mask panel) | Show Flood Overlay (colour-blended overlay panel).

8. Common Errors and Fixes

Error: No module named 'configs'

Fix: You are running a script from the wrong directory. Always run commands from the project root:

```
cd C:\Users\Test1\Desktop\flood_victim_detection_v3
```

Error: FileNotFoundError for dataset paths

Fix: Your PROJECT_ROOT in configs\config.py does not match where your datasets live. Open the file and update the path.

```
PROJECT_ROOT = r"C:\Users\YourName\Desktop\your_dataset_folder"
```

Error: No checkpoint found at checkpoints/flood_classifier_best.pth

Fix: Phase 2 training has not been completed yet. The dashboard will still load but predictions will be random.

```
python classification\training\train_classifier.py
```

Error: No checkpoint found at checkpoints/best_model.pth

Fix: Phase 3 training has not been completed yet.

```
python training\train.py
```

Error: ModuleNotFoundError: No module named 'ultralytics'

Fix: The virtual environment is not active, or requirements were not installed.

```
venv\Scripts\activate  
pip install -r requirements.txt
```

Error: YOLO model download fails

Fix: YOLOv8n needs internet access on first run to download yolov8n.pt (6 MB). If on a restricted network, download it manually from <https://github.com/ultralytics/assets/releases> and place it in the project root.

Error: Streamlit port already in use

Fix: Port 8501 is occupied by another process. Use a different port:

```
streamlit run streamlit_app\app.py --server.port 8502
```

9. Complete Run Order — Quick Reference

Run all commands from the project root folder with the virtual environment active. Steps 1 to 8 are one-time setup. Steps 9 and 10 produce the trained checkpoints. Step 11 is what you run every time you want to use the system.

Step	Action
Step 1	Install Python 3.10 or 3.11 from python.org
Step 2	Extract zip to Desktop
Step 3	Set up dataset folders at correct paths
Step 4	cd flood_victim_detection_v3
Step 5	python -m venv venv
Step 6	venv\Scripts\activate
Step 7	pip install -r requirements.txt
Step 8	python utils\verify_dataset.py
Step 9	python data\build_metadata.py
Step 10	python data\create_splits.py
Step 11	python classification\training\train_classifier.py
Step 12	python training\train.py
Step 13	streamlit run streamlit_app\app.py

10. CPU Performance Expectations

Task	Expected Speed
Flood Classification	20 to 40 ms per image
Flood Segmentation	120 to 250 ms per image
YOLO Person Detection	60 to 120 ms per image
Full Pipeline (image)	300 to 600 ms per image
Video Processing	2 to 5 FPS on CPU

11. Risk Scoring Formula

```
Risk Score = 0.40 x overlap
+ 0.30 x water_density
+ 0.20 x segmentation_confidence
+ 0.10 x body_visibility
```

Score Range	Risk Level	Bounding Box Colour
Score >= 0.75	CRITICAL	Red
Score >= 0.50	HIGH	Orange
Score >= 0.25	MEDIUM	Yellow
Score < 0.25	LOW	Green